

Semester Reflection Report: Moving Towards Contemporary Enterprise Development Focus Area: Browser-Based Data Processing & Live Analytics Interfaces

Over the course of our final-year project, my role grew beyond what I initially expected. I found myself taking ownership of the data reporting side of our car wash management system specifically, a financial monitoring dashboard built for the Witbank branch. My academic background had been rooted in Java EE 7, where generating reports meant leaning on server-side tools, writing heavy database queries, or pushing output through JSPs using frameworks like JasperReports. Stepping into a React environment challenged everything I thought I knew about where logic belongs in an application and completely changed how I think about building software.

Moving Business Logic to the Browser

In the older Java EE style of development, the browser was never trusted to do anything meaningful. It simply received pre-built HTML from the server and displayed it. Whenever a manager needed revenue figures or a breakdown of services, all of that processing happened on the server queries ran, objects were populated, and the results were sent down to a template that rendered everything statically.

Working on the Reports component in React flipped this entirely. My job was to call two separate endpoints on our Spring Boot API one returning the current booking queue and another returning that day's running financial total. Rather than waiting for the server to crunch those numbers, I wrote the logic directly in JavaScript, running inside the user's browser. Looping through the booking data, I categorized each entry by wash type whether it was a Standard wash, a Full Valet, or an Engine clean and calculated the projected income on the fly. It felt strange at first, putting that kind of responsibility on the frontend, but it made sense once I saw the results. Modern devices are more than capable of handling these calculations, and keeping that load off the server makes the whole system more capable of scaling.

Working with Asynchronous Requests

Something that took real effort to get comfortable with was managing multiple network calls happening at the same time. Java EE had sheltered me from this operations ran one after another, each waiting for the previous one to finish before anything else moved. JavaScript does not work that way, and at first that was disorienting.

The `async/await` pattern, combined with the Axios library, gave me a clean way to write code that fetched from both endpoints without the response handling becoming a tangled mess. What also became very clear was that network communication is unpredictable. The React app

and the Spring Boot backend run on different ports, and in a real production setup they could be on entirely separate machines. Delays happen. Requests fail. Wrapping everything in a try/catch block meant that when something went wrong on the network side, the dashboard stayed standing rather than crashing in front of a manager trying to run their shift.

Handling a More Involved State Structure

Before this project, my experience with React state was limited to fairly simple values, a flag here, a list there. The Reports component needed something more involved. I had to track the total vehicle count, the revenue projection, the daily total, and a nested object breaking down numbers by wash type all updated together and all reflected immediately in the interface.

In Java EE, keeping that kind of data available across a session meant storing it in the HttpSession object, which carries its own risks around server memory if sessions are not properly managed. In React, all of it lives on the client side, which felt lighter but came with its own discipline requirement. The biggest adjustment was learning that you cannot simply reach into state and change a value. React requires you to replace the entire state object with a new one. Coming from an object-oriented background where you call a setter method and move on, this felt unnecessarily strict until I understood why. Replacing state rather than mutating it prevents a whole category of subtle bugs where parts of the interface fall out of sync with each other.

Building Charts Without a Charting Library

One of the more satisfying parts of this project was constructing the wash-type bar charts. In past modules, adding any kind of chart meant pulling in a large Java library and spending time configuring it just to display something basic. The result was usually functional but rigid.

In React, I wrote a simple helper function that worked out what percentage of the total queue each wash type represented. That percentage was then applied directly as an inline CSS width on a plain div element. No external library. No complicated configuration. Just a calculated number plugged into a style property. When the polling mechanism fetched fresh data every five seconds and updated the state, React identified exactly which parts of the interface had changed and updated only those no flicker, no full-page reload. Watching those bars adjust smoothly in real time, reflecting actual live data, was a genuinely rewarding moment.

Keeping the Dashboard Alive Responsibly

Keeping data current meant setting up a repeating interval that triggered a fresh data fetch every five seconds. This introduced a concern that goes beyond simply making things work it raised questions about what happens when the component is no longer needed. If the polling

keeps running after a user navigates away, it wastes browser resources and keeps hitting the server for no reason.

The solution was a cleanup function returned from the `useEffect` hook, which cancels the interval the moment the component is removed from the page. This reinforced something important: in a decoupled system, the frontend developer carries genuine responsibility for how resources are used. Java's runtime and servlet containers handle a lot of that automatically, but React expects you to manage it yourself.

Closing Thoughts

This project represented a real turning point in how I understand software development. I came in thinking of the frontend as a passive display layer and left with a completely different perspective. A well-built React application is a capable piece of software in its own right it processes data, manages its own state, communicates across a network, and handles failures gracefully. I leave this semester with practical skills I am confident will carry into professional work.